

Project Architecture

1. High-Level Workflow

This project enables real-time object detection for visually impaired users using hand gestures and provides audio feedback for recognized objects.

Step 1: Capture Live Video

The camera captures a real-time video stream. Frames are continuously processed to detect hand gestures and objects.

Step 2: Hand Tracking & Gesture Recognition

The MediaPipe Hands model tracks the user's hand. The index finger's position is extracted to determine if the user is pointing.

Step 3: Object Detection (YOLOv8)

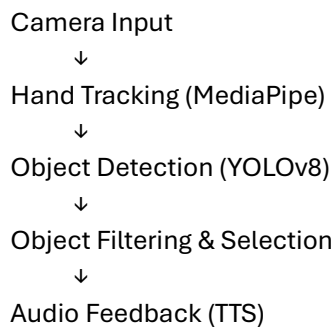
The YOLOv8 model detects objects in the frame. Only objects above the fingertip are considered. Small objects are also detected with fine-tuned thresholds. The model excludes objects inside or too close to the hand bounding box.

Step 4: Object Selection & Audio Feedback

If a detected object is stable for 2 seconds, it is confirmed. The object's name is spoken using a Text-to-Speech (TTS) engine. The system waits before detecting a new object to avoid redundancy.

2. System Architecture Diagram

High-level system architecture:



3. Module Breakdown

Hand Tracking Module (`handtracking.py`)

Uses MediaPipe Hands to detect the index finger position. Determines if the user is pointing based on finger angles. Applies smoothing to stabilize the fingertip position.

Object Detection Module (`object_detection.py`)

Uses YOLOv8 for real-time object detection. Filters objects close to the hand or below the fingertip. Uses Non-Maximum Suppression (NMS) to remove duplicate detections.

Object Selection Module (integration.py)

Matches the pointed fingertip with detected object bounding boxes. Only selects objects within the fingertip area. Ensures the object is stable before confirming detection.

Audio Feedback Module (audio_feedback.py)

Uses pyttsx3 (Text-to-Speech) to announce object names. Runs in a separate thread to avoid blocking detection. Prevents repetitive speech by adding a 2-second delay.

Main Integration (main.py)

Captures video from the camera. Runs hand tracking & object detection in separate threads. Displays real-time feedback using OpenCV. Stops when the user presses 'q'.

4. Technologies Used

Machine Learning Models

YOLOv8 – Object Detection (CNN-based)

MediaPipe Hands – Hand & Finger Tracking

pyttsx3 – Text-to-Speech Engine

Libraries & Frameworks

OpenCV – Video Processing & Visualization

Ultralytics – YOLOv8 Model Wrapper

NumPy – Image Processing & Computation

Threading – Asynchronous Processing

5. Requirements.txt (Dependencies List)

ultralytics

mediapipe

opencv-python

numpy

pyttsx3

6. Optimizations

Optimization Techniques

Uses threading for real-time performance.

Implements smoothing filters to stabilize finger tracking.

Uses Non-Maximum Suppression (NMS) to refine detections.